Project of: Marco Colangelo (Student ID: 67045A)
Algorithms for Massive Datasets, Academic Year 2024/25
Master's Degree in Computer Science, University of Milan

Finding Similar Amazon Books Reviews using LSH, MinHashing and Jaccard Similarity

September 4, 2025

1 Introduction

This report contains an overall on the work carried out for the final project of the course of Algorithms for Massive Datasets. The professor in charge of the course is Professor Dario Malchiodi and the course was taught at the University of Milan in the second semester of the 2024/25 academic year.

Among the various assignments for the project, all consistent with the topics presented in the course, this work focused on the problem of **finding similar items**. More specifically, the work focused on the implementation of a specific procedure presented in the course for finding similar items, which combines MinHashing and Locality-Sensitive Hashing (LSH) techniques to find similar items using Jaccard Similarity. These techniques will be examined in detail later in this report, particularly their theoretical foundations and how they have been implemented and parameterized.

For the project, the dataset used is Amazon Books Reviews, made available on Kaggle and containing information about various books and user reviews associated with those books. The version used for this project is the version 1, dated 14 September 2022, and the dataset is not pre-loaded but downloaded at runtime. The project focused on the task of finding similar reviews, using the full text of the reviews.

This project was built on Google Colab and is intended to run on a distributed computing environment. For this purpose, the PySpark library, which provides a Spark API for Python, was used.

2 The Amazon Books Reviews dataset

The Amazon Books Reviews dataset, in the version used (version 1 of 2022), has a total size of approximately 3 gigabytes and consists of two tables:

- `books_rating`: has a total size of approximately 2.8 gigabytes and contains 3 million reviews. The table has several columns: however, both for the purposes of the work performed and to reduce the data loaded into memory, the only columns retained were:
  - `ID`: Unique ID for a book
  - `User_id`: The ID of the user who wrote the review
  - `Title`: Title of the book
  - `Review/text`: Contains the entire text of the review

  For this dataset, the primary key (i.e. what characterizes a review) is made up of the pair of values `(ID, User_id)`: this is the reason those columns were necessary.

- `books_data`: has a size of approximately 180 megabytes and contains information for approximately 200000 books. This table will not be considered, as the work performed focused on book reviews.

This dataset is downloaded at runtime and, as will be shown later, will be sampled randomly to ensure reasonably low execution times and memory requirements.

3 Manipulation of the dataset

The following section will explain how the dataset was manipulated for the purposes of the work performed, both in terms of subsampling and preprocessing applied.

3.1 Dataset subsampling

As shown previously, the `books_rating` dataset used is large, which would significantly increase execution time on Google Colab, as a true computing cluster isn't available. For this reason, the `books_rating` dataset was sampled, randomly selecting 30000 elements.

3.2 Dataset preprocessing

First, only the `ID, Title, User_id` and `review/text` columns were selected from the sample. The sample was then filtered to remove all rows with empty or null `review/text` columns, thus eliminating reviews of this type. The resulting sample has the following structure:

Id	User_id	Title	review/text
0826414346	A3UH4UZ4RSVO82	Dr. Seuss: American Icon	If people become the books the
0595344550	AVRN9E6X9I6KF	Whispers of the Wicked Saints	I thought this book was brilli
0847822982	A3UYWQ5AHHQHGC	Portraits of the Presidents: The National Portrait Gallery	I had very high expectations f
B0007DVHU2	A1LJX75GT1KKLT	Treat yourself to life	Dr. Barker opened the First Ch
0806525428	A1DHA0GWSWQRG0	The New Conspiracy Reader: From Planet X to the War on Terrorism-What You Really Don't Know	I remember getting this book s
006000486X	A1PQJ6IPEQB3CD	Tess and the Highlander	When I got this book, I wasn't
0781810698	A1HHC466MCO9QE	Beginner's Yoruba (Hippocrene Beginner's Series)	This book contains a great amo

Figure 1: Structure of the sample used

However, it should be noted that no row in the sample ever contained an empty or null review, but the filter was still applied to provide for the possibility of handling such cases.

The second preprocessing step applied was some kind of normalization of the `review/text` column for all rows in the sample. This normalization involved three steps:

1. Converting the **entire text to lowercase**, so that a lowercase letter and an uppercase letter are not considered different (for example, "House" and "home" are the same word, but they do not have the same letters).

2. **Removing punctuation marks and non-alphanumeric characters** as these would generate inconsistent shingles. With regard to **spaces**, it was decided to leave at least one space between words (if these are present, of course) and, when punctuation marks and non-alphanumeric characters were removed, these have been replaced with a space.

3. **Removing all stop words** from the text, such as the, an, and, or and so on. The reason why they were removed is because they are rather frequent words in sentences and therefore do not provide a particularly informative contribution.

After this preprocessing, a `review/text` occurrence went from this

```
If people become the books they read and if "the child is father to the man," then Dr. Seuss (Theodor Seuss Geisel)
```

Figure 2: review/text before normalization

to this

```
people become books read child father man dr seuss theodor seuss
```

Figure 3: review/text after normalization

And that is all for the preprocessing phase. In the next section, some theoretical foundations of the problem of finding similar elements will be recalled and how these have been applied to produce the obtained results.

4 Jaccard similarity, MinHashing and LSH to find similar items

4.1 Jaccard similarity

The course presented the problem of finding similar elements considering textual documents, but clearly the elements can also be non-textual. One metric used to determine how much two textual documents are similar is Jaccard similarity, defined as

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1] \tag{1}$$

where A and B are the two textual documents. The closer it is to 1, the more similar the documents are.

4.2 Shingling of text for Jaccard similarity

A preprocessing technique for applying Jaccard similarity is **shingling the text content**: a shingle is a k-gram, a set of k atomic information, such as the individual characters that make up a word or sentence. The choice of the k parameter is very important, because if it is chosen too low, for example, it would cause many documents to share the same k-shingles and therefore appear similar, even if they actually are not. A general rule is the following:

> *k should be chosen large enough so that the probability of any shingle appearing in a given document is low.*

For the project, the **chosen value for k was 5**. This is because the length of the review texts in the dataset varies greatly, with both fairly short reviews and much longer ones. Choosing an ideal k parameter for very long reviews (around 8-9) would have penalized the shorter ones. For this reason, k = 5 was chosen as the value for the shingle length, as it represents a good compromise between shingle specificity and the quantity of shingles generated even for the shortest reviews.

Let's look at an example of transforming the text of a review into 5-character shingles. Returning to the previous example shown in Figures 2 and 3, some of the generated shingles would be as follows:

```
shingle='peopl'
shingle='eople'
shingle='ople '
shingle='ple b'
shingle='le be'
shingle='e bec'
shingle=' beco'
shingle='becom'
shingle='ecome'
shingle='come '
```

After the generation of the shingles, the next step is to create an encode for them. A simple approach is using hash functions. For the work done, an hash function

$$h : \{\text{shingle}\} \rightarrow \{0 \dots 2^{64}\} \tag{2}$$

has been defined. This hash function takes as input a shingle and produces as output an integer in the range between 0 and $2^{64} - 1$. The following figure

```
shingle='peopl'  → hash=16150579320486149805
shingle='eople'  → hash=15843308918401268826
shingle='ople '  → hash=9010682342196415709
shingle='ple b'  → hash=1707704388660126712
shingle='le be'  → hash=10859088717832694993
shingle='e bec'  → hash=6889281743208781064
shingle=' beco'  → hash=2756774730868465654
shingle='becom'  → hash=16195861718255264827
shingle='ecome'  → hash=17663458723202126858
shingle='come '  → hash=1320136042484977405
```

shows an example of this hash function applied to some of the shingles generated in the previous step.

In the lecterature, to visualize the relation between shingles a characteristic matrix (let's call it CM) can be constructed. This is a $s \times n$ matrix where s is the number of possible shingles and n is the number of documents and is defined as:

$$CM_{ij} = \begin{cases} 1 & \text{if shingle i is contained in document j} \\ 0 & \text{otherwhise} \end{cases} \tag{3}$$

There is a problem: this matrix is huge and sparse so we have to compress it, if we want to calculate it. This is where MinHashing comes in handy.

4.3 MinHashing

MinHashing is a data compression method that allows us to efficiently estimate Jaccard similarity between sets without having to work directly with the full sets or characteristic matrix. The steps it requires are the following:

1. A row permutation of the characteristic matrix choosing a certain hash function.

2. $\forall$ documents, scan its document column in the matrix until the fist 1, and return the corresponding shingle row.

3. Construction of the Signature Matrix (let's call it SM), which columns are intended as signatures and define a document. A signature is an array of values, each corresponding to the output of an hash function applied after a permutation of the rows. It's a compact representation of the characteristic matrix.

In the work done, **the number H of independent hash functions choosen was 128**, which has been resulted as a good compromise between the accuracy of the Jaccard similarity and the computational cost required. Every hash function is in the form of

$$h_i(x) = (a_i \cdot x + b_i) \text{ mod p} \tag{4}$$

where a_i and b_i are casual values with fixed seed for reproducibility, p is a large prime number and x is the encode of the shingle. The reasons behind the choise of this hash function are several: first of all, is very simple to calculate, which is ideal for large datasets. Second, picked a and b randomly, the probability that two distinct values x_1 and x_2 collides is limited. Finally, for MinHashing a lot of independent hash functions are required and varying a and b makes sure that each hash function behave like a different permutation in the space of the shingles.

Using these hash functions allows us to exploit a very important theorem that further reduces computational cost. The hash functions used are random functions: that is, each time the characteristic matrix is observed, a permutation is randomly (and uniformly) chosen and the MinHashing is calculated. We define as E the event $H(D_1) = H(D_2)$ where D_1 and D_2 are two documents: this means that in the permuted matrix, for a given hash function h, the two documents D_1 and D_2 will have the same value k.

It can be shown that

$$P(E) = J(D_1, D_2) \tag{5}$$

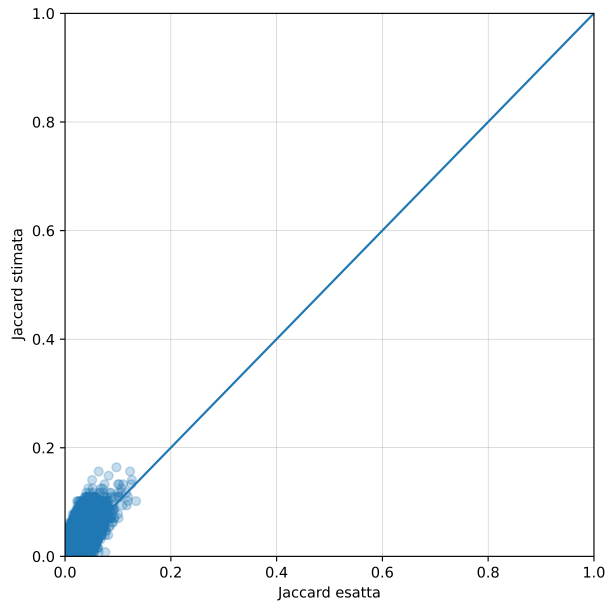
that is, the number of favorable cases in which $H(D_1) = H(D_2)$, among all possible ones, is exactly the Jaccard similarity of the two documents.

This theorem allows us to estimate the Jaccard similarity between two documents D_1 and D_2 by simply calculating the ratio between the rows of the matrix where $D_1 = D_2$ and all the rows of the matrix. Formally

$$\hat{J}(A, B) = \frac{\text{Lines where } D_1 = D_2}{\text{All lines}} \tag{6}$$

This is how Jaccard similarity has been calculated to find similar reviews.

We see the satisfaction of this theorem in the work performed: 3000 elements were randomly selected from the sample, and for each pair in this sample, the exact Jaccard similarity was calculated on the set of their shingles and the estimated Jaccard similarity using the signatures produced by MinHashing. Subsequently, the RMSE was calculated to measure the average error between the estimated and exact Jaccard similarity, and a correlation graph was constructed where each point corresponds to a pair of documents, the x-axis corresponding to the exact Jaccard similarity, and the y-axis corresponding to the estimated Jaccard similarity. The RMSE obtained was approximately 0.0114, a value close to 0, indicating a low error in the calculated estimate. Let's now analyze the constructed graph:



It can be seen how the values are distributed along the bisector, indicating that the estimate is very close to the true value. The reason why most of the points are located in the lower left indicates that the Jaccard similarity values are not particularly high, but they are compatible with the sample size used.

Speaking about the Signature Matrix, in the work done this data structure was implemented using Spark's Resilient Distributed Datasets (RDDs). This was because the signature matrix would have been large anyway, and given the desire to leverage a distributed computing environment, this was the preferred approach. Specifically, here's what was done:

1. For each document, a min-hash of length H = 128 was calculated using the previously specified hash function.

2. These min-hashes were saved as an array of length H and associated with the encoded document ID. This resulted in an RDD of pairs of the form `(document_ID, signature_vector)`, each of which can be thought of as a sparse column of the signature matrix. This is then used in LSH for the banding phase and for matching signatures to estimate Jaccard similarity.

There is a practical problem: computing all the possible permutations of a set is time-consuming, but we can use hash functions to simulate the effect of a random permutation. Time complexity, however, remains prohibitive: having 1 milion documents, each of which consists of 250 4 bytes words, we would have a total of 1 gigabyte. If we suppose that each document is checked in 1 μs, this would require about 5-6 days of total computational time, which is not the ideal for large datasets. For this reason, the signature matrix construction approach has not been considered in this work, instead applying a combination of MinHahsing and LSH, which will be presented in the next section.

4.4 Locality-Sensitive Hashing (LSH)

Locality-sensitive hashing is a technique that significantly reduces time complexity by drastically reducing the number of document pairs to compare, selecting only a subset of candidates that are highly likely to be similar, or, in other words, with a Jaccard similarity above a certain threshold. This technique hashes elements multiple times, making it more likely that similar elements will be hashed. Candidate pairs are those that are hashed to the same bucket, and Jaccard similarity is calculated only for these.

The idea is to divide the signature matrix into b bands of r rows each, and each band is associated with a hash function that takes a vector of r integers and hashes them to buckets. This means that if two documents are similar and should be considered potential candidates, then at least one of their bands must be identical. It can be shown that, by defining the event E = "The two documents have the same projection (or sub-signature) in at least one band," then

$$P(E) = 1 - (1 - s^r)^b := p(s) \tag{7}$$

By plotting $p(s)$, we can see how it always takes the form of a sigmoid, whose inflection point depends strictly on the choice of parameters b and r and is a threshold beyond which two documents become potential candidates for similarity. We call this point s^*: by calculating the second derivative of $p(s)$, we can find this point, which translates to

$$s^* = (\frac{1}{b})^{\frac{1}{r}} \tag{8}$$

which will therefore be the minimum similarity threshold for considering two documents similar.

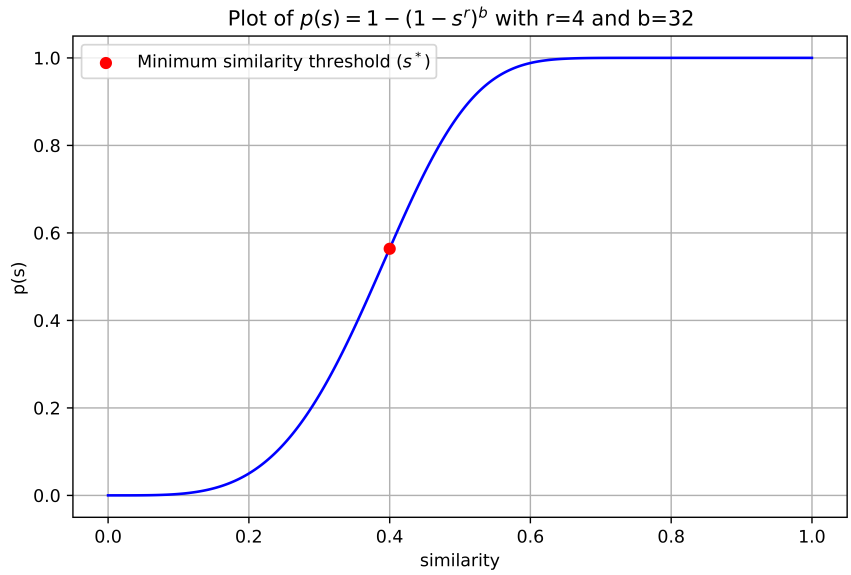
We therefore have a formula for finding the minimum similarity threshold, however, it depends on the choice of parameters b and r. To determine these two parameters and then determine the minimum similarity threshold, it is necessary to solve this simple system:

$$\begin{cases} br = n \\ t \approx (\frac{1}{b})^{\frac{1}{r}} \end{cases} \tag{9}$$

where n is the length of the MinHashing signatures, which in our case is equal to 128.

For the work done it has been decided a **minimum similarity threshold t equal to about 0.4**: since n = 128, to obtain $t \approx 0.4$ we set $r = 4$ **and** $b = 32$**.**

The following figure



shows the graph for the function $p(s)$ with b=32 and r=4 as parameters. The red dot is approximately 0.4 and is the minimum similarity threshold.

At this point we have all the elements necessary to define a specific procedure to find similar reviews, using Jaccard similarity as a similarity metric and using MinHashing and LSH to reduce the computational complexity with all the necessary precautions provided by Spark to scale even to larger datasets having a real cluster available.

1. Pick a value of k (in our case k = 5) and $\forall$ document construct a set of k-shingles.

2. Pick a length n for the Min-Hash signatures (in our case n = 128), and construct the signature matrix having dimensions $n \times D$, where D is the number of reviews.

3. Choose a threshold t (in our case t = 0.4) that defines how similar two documents have to be in order for them t be considered as "candidates" for the actual similarity computation. Then, pick the values of b (bands) and r (rows within a band) such that $br = n$ and $t \approx (\frac{1}{b})^{\frac{1}{r}}$ (in our case, b = 4 and r = 32).

4. Construct the candidate pairs by applying LSH.

5. Examine each candidate pair's signatures and determine whether the fraction of components in which they agree is at least t.

This is the procedure that has been implemented in the work done. In the next section the results obtained will be presented and commented on.

5 Results

At this point, having examined the techniques used in the project from both a theoretical perspective and the choice of parameters, several runs of the written code were performed

and the results were combined. This is because subsampling is performed randomly, so different results can be obtained with each run.

These are the similar reviews obtained (remember, the minimum similarity threshold was set to 0.4). Only a few of the reviews are shown, as some are particularly long.

- Review 1:
- Review 2:
- Jaccard similarity:

. . . . . .

- Review 1:
- Review 2:
- Jaccard similarity:

. . . . . .

- Review 1:
- Review 2:
- Jaccard similarity: